Binder Diver Engine Specification v2.0

Target Audience: Claude Code Agent / LLM Developer Engine
Objective: Implement a flexible Retail Layout Logic Matrix capable of routing Scryfall card variants into physical store containers, featuring customizable group bindings and modular boolean toggles.

1. Variant Data Mapping & Weighted Pricing Tiers

Variant properties from the Scryfall API greatly skew sorting logic. Premium card types (Foils, Extended Art, Showcases) can elevate a standard bulk card into display-case pricing. The internal logic must isolate the specific *printing price* rather than the base non-foil oracle price.

```
# Variant Weighting Matrix Structure
card_variant = {
    "name": "Smothering Tithe",
    "set_code": "rna",
    "is_foil": True,
    "border_color": "borderless",
    "finishes": ["foil"],
    "scryfall_prices": {
        "usd": "12.50",
        "usd_foil": "28.00",
        "usd_etched": None
    }
}

# Determine Active Market Value based on specific card finish
def get_active_price(variant):
    prices = variant.get("scryfall_prices", {})
    if variant.get("is_foil") and prices.get("usd_foil"):
        return float(prices["usd_foil"])
    return float(prices.get("usd", 0.0))
```

2. Modular Store Profile Configuration (The Questionnaire Schema)

To keep the initial setup lightweight, stores are modeled using basic Boolean switches and value thresholds. This allows the system to seamlessly route cards without hardcoded spatial dependencies.

```
store_profile_schema = {
    "store_id": "lgs_001",
    "settings": {
        "uses_display_case": True,
        "display_case_threshold": 25.00,
        "uses_premium_color_binders": True,
        "premium_binder_threshold": 5.00,
        "uses_dedicated_commander_containers": True,
        "uses_set_binders": True,
        "set_binder_age_cutoff_years": 3
    }
}
```

3. Advanced Feature: Dynamic Binder Groupings

Many LGS entities bundle highly related sets together into single physical folders to save floor space. For example, matching a main premier set with its complementary *The List*, *Special Guests*, or standard commander subset cards into one book. The script below implements a tag-matching catch mechanism.

```
# Flexible grouping dictionary configuration
store_binder_groups = {
    "group_foundations": {
        "display_name": "Foundations Binder",
        "matching_sets": ["fdn", "fbc", "plst"], # Foundations, Box Toppers, The List
    },
    "group_strixhaven": {
        "display_name": "Strixhaven Binder",
        "matching_sets": ["stx", "c21", "sta"]  # Main set, C21, Mystical Archive
    }
}

def check_binder_groups(set_code, groups_config):
    for group_id, meta in groups_config.items():
        if set_code.lower() in meta["matching_sets"]:
            return f"Look in grouped folder: {meta['display_name']}"
    return None
```

4. Combined Cascading Layout Evaluation Algorithm

The primary algorithm executes deterministically, filtering inputs through the store configuration ruleset.

```
def route_card_location(card, variant, store, groups):
    price = get_active_price(variant)
    set_code = card.get("set_code", "").lower()

    # Tier 1: Security Display Check
    if store["settings"]["uses_display_case"] and price >= store["settings"]["display_case_thresh
old"]:
        return "Display Case (Premium Section)"

    # Tier 2: High Value Color Folder
    if store["settings"]["uses_premium_color_binders"] and price >= store["settings"]["premium_bi
nder_threshold"]:
        return f"High-Value Binder -> Sorted by Color ({card.get('color_identity')})"

    # Tier 3: Explicit Binder Group Interception
    grouped_folder = check_binder_groups(set_code, groups)
    if grouped_folder and store["settings"]["uses_set_binders"]:
        return grouped_folder

    # Tier 4: Format-Specific Buckets
    if store["settings"]["uses_dedicated_commander_containers"] and "commander" in card.get("lega
lities", {}):
        if card.get("is_cmd_supplement") or "snc" in set_code:
            return "Dedicated Commander Box/Binder"

    # Tier 5: Fallback Bulk Rows
    return f"Bulk Storage Rows -> Alphabetical A-Z Box or Box Color: {card.get('color_identity')}
"
```